

- PARSEC (Princeton Application Repository for Shared Memory Computers), 556
- Pass transistor, A-62
- PCI-Express (PCIe), 553, B-8, OL6.9-2
- PC-relative addressing, 118, 120
- Peak floating-point performance, 558
- Pentium bug morality play, 244
- Performance, 28–40
 - assessing, 28
 - classic CPU equation, 36–40
 - components, 38
 - CPU, 33–35
 - defining, 29–32
 - equation, using, 36
 - improving, 34–35
 - instruction, 35–36
 - measuring, 33–35, OL1.12-10
 - program, 39–40
 - ratio, 31
 - relative, 31–32
 - response time, 30–31
 - sorting, B-54–55
 - throughput, 30–31
 - time measurement, 32
- Personal computers (PCs), 7
 - defined, 5
- Personal mobile device (PMD)
 - defined, 7
- Petabyte, 6
- Physical addresses, 442
 - mapping to, 442–443
 - space, 533, 535
- Physically addressed caches, 458
- Pipeline registers
 - before forwarding, 320
 - dependences, 319
 - forwarding unit selection, 323
- Pipeline stalls, 291
 - avoiding with code reordering, 291
 - data hazards and, 324–328
 - insertion, 326
 - load-use, 329
 - as solution to control hazards, 293
- Pipelined branches, 331
- Pipelined control, 311–315. *See also* Control
 - control lines, 311, 312
 - overview illustration, 327
 - specifying, 312
- Pipelined datapaths, 297–315
 - with connected control signals, 315
 - with control signals, 311–315
 - corrected, 307
 - illustrated, 300
 - in load instruction stages, 307
- Pipelined dependencies, 317
- Pipelines
 - branch instruction impact, 329
 - effectiveness, improving, OL4.16-4–4.16-5
 - execute and address calculation stage, 301, 303
 - five-stage, 285, 301, 309
 - graphic representation, 290, 307–311
 - instruction decode and register file
 - read stage, 300, 303
 - instruction fetch stage, 301, 303
 - instructions sequence, 325
 - latency, 297
 - memory access stage, 301, 303
 - multiple-clock-cycle diagrams, 308
 - performance bottlenecks, 353
 - single-clock-cycle diagrams, 308
 - stages, 285
 - static two-issue, 345
 - write-back stage, 301, 305
- Pipelining, 12, 283–297
 - advanced, 354–355
 - benefits, 283
 - control hazards, 292–293
 - data hazards, 289
 - exceptions and, 338–342
 - execution time and, 297
 - fallacies, 366–367
 - hazards, 288
 - instruction set design for, 288
 - laundry analogy, 284
 - overview, 283–297
 - paradox, 285
 - performance improvement, 288
 - pitfall, 366–367
 - simultaneous executing instructions, 297
 - speed-up formula, 285
 - structural hazards, 288, 305
 - summary, 296
 - throughput and, 297
- Pitfalls. *See also* Fallacies
 - address space extension, 493
 - arithmetic, 242–245
 - associativity, 492
 - defined, 49
 - GPUs, B-74–75
 - ignoring memory system behavior, 491
 - memory hierarchies, 491–495
 - out-of-order processor evaluation, 492
 - performance equation subset, 50–51
 - pipelining, 366–367
 - pointer to automatic variables, 171
 - sequential word addresses, 171
 - simulating cache, 491
 - software development with
 - multiprocessors, 570
 - VMM implementation, 495
- Pixel shader example, B-15–17
- Pixels, 18
- Pointers
 - arrays *versus*, 146–150
 - frame, 106
 - global, 106
 - incrementing, 148
 - Java, OL2.15-26
 - stack, 101, 105
- Polling, OL6.9-8
- Pop, 101
- Power
 - clock rate and, 40
 - critical nature of, 53
 - efficiency, 354–355
 - relative, 41
- PowerPC
 - algebraic right shift, D-33
 - branch registers, D-32–33
 - condition codes, D-12
 - instructions, D-12–13
 - instructions unique to, D-31–33
 - load multiple/store multiple, D-33
 - logical shifted immediate, D-33
 - rotate with mask, D-33
- Precise interrupts, 342
- Prediction, 12
 - 2-bit scheme, 333
 - accuracy, 333
 - dynamic branch, 331–333
 - loops and, 333–334
 - steady-state, 333
- Prefetching, 496, 560
- Primitive types, OL2.15-26
- Procedure calls
 - preservation across, 106
- Procedures, 100–110
 - compiling, 102
 - compiling, showing nested procedure
 - linking, 102–104
 - execution steps, 100

- frames, 106
 - leaf, 104
 - nested, 104–106
 - recursive, 108
 - for setting arrays to zero, 147
 - sort, 140–145
 - strcpy, 112–113
 - string copy, 112–113
 - swap, 138
 - Process identifiers, 460
 - Process-level parallelism, 516
 - Processors, 254–368
 - as cores, 43
 - control, 19
 - datapath, 19
 - defined, 17, 19
 - dynamic multiple-issue, 343
 - multiple-issue, 343
 - out-of-order execution, 355, 430
 - performance growth, 44
 - ROP, B-12, B-41
 - speculation, 344–345
 - static multiple-issue, 343, 345–349
 - streaming, B-34
 - superscalar, 349, 531–532, OL4.16-5
 - technologies for building, 24–28
 - two-issue, 346
 - vector, 523–524
 - VLIW, 345
 - Product, 192
 - Product of sums, A-11
 - Program counters (PCs), 263
 - changing with conditional branch, 334
 - defined, 101, 263
 - exception, 459, 461
 - incrementing, 263, 265
 - instruction updates, 300
 - Program performance
 - elements affecting, 39
 - understanding, 9
 - Programmable array logic (PAL), A-77
 - Programmable logic arrays (PLAs)
 - component dots illustration, A-16
 - control function implementation, C-7, C-20–21
 - defined, A-12
 - example, A-13–14
 - illustrated, A-13
 - ROMs and, A-15–16
 - size, C-20
 - truth table implementation, A-13
 - Programmable logic devices (PLDs), A-77
 - Programmable ROMs (PROMs), A-14
 - Programming languages. *See also* specific languages
 - brief history of, OL2.22-7–2.22-8
 - object-oriented, 150
 - variables, 67
 - Programs
 - assembly language, 129
 - Java, starting, 136–137
 - parallel processing, 516–523
 - starting, 128–137
 - translating, 128–137
 - Propagate
 - defined, A-39
 - example, A-44
 - super, A-40
 - Protected keywords, OL2.15-21
 - Protection
 - defined, 442
 - implementing, 459–460
 - mechanisms, OL5.17-9
 - VMs for, 438
 - Protection group, OL5.11-5
 - Pseudo MIPS
 - defined, 246
 - instruction set, 248
 - Pseudoinstructions
 - defined, 129
 - summary, 130
 - Pthreads (POSIX threads), 556
 - PTX instructions, B-31, B-32
 - Public keywords, OL2.15-21
 - Push
 - defined, 101
 - using, 104
- ## Q
- Quad words, 158
 - Quicksort, 425, 426
 - Quotient, 198
- ## R
- Race, A-72
 - Radix sort, 425, 426, B-63–65
 - CUDA code, B-64
 - implementation, B-63–65
 - RAID. *See* Redundant arrays of inexpensive disks (RAID)
 - RAM, 9
 - Raster operation (ROP) processors, B-12, B-41, B-50–51
 - fixed function, B-41
 - Raster refresh buffer, 18
 - Rasterization, B-50
 - Ray casting (RC), 568
 - Read-only memories (ROMs), A-14–16
 - control entries, C-16–17
 - control function encoding, C-18–19
 - dispatch, C-25
 - implementation, C-15–19
 - logic function encoding, A-15
 - overhead, C-18
 - PLAs and, A-15–16
 - programmable (PROM), A-14
 - total size, C-16
 - Read-stall cycles, 413
 - Read-write head, 395
 - Receive message routine, 545
 - Recursive procedures, 108. *See also* Procedures
 - clone invocation, 104
 - Reduced instruction set computer (RISC)
 - architectures, D-2–45, OL2.22-5, OL4.16-4. *See also* Desktop and server RISCs; Embedded RISCs
 - group types, D-3–4
 - instruction set lineage, D-44
 - Reduction, 535
 - Redundant arrays of inexpensive disks (RAID), OL5.11-2–5.11-8
 - history, OL5.11-8
 - RAID 0, OL5.11-4
 - RAID 1, OL5.11-5
 - RAID 2, OL5.11-5
 - RAID 3, OL5.11-5
 - RAID 4, OL5.11-5–5.11-6
 - RAID 5, OL5.11-6–5.11-7
 - RAID 6, OL5.11-7
 - spread of, OL5.11-6
 - summary, OL5.11-7–5.11-8
 - use statistics, OL5.11-7
 - Reference bit, 450
 - References
 - absolute, 131
 - types, OL2.15-26
 - Register, 31, 74, 102, 175
 - Register addressing, 120
 - Register allocation, OL2.15-11–2.15-13

- Register files, A-49, A-53–55
 - defined, 264, A-49, A-53
 - in behavioral Verilog, A-56
 - single, 269
 - two read ports implementation, A-54
 - with two read ports/one write port, A-54
 - write port implementation, A-55
 - Register-memory architecture, OL2.22-3
 - Registers, 156, 157–158
 - architectural, 336–342
 - base, 69
 - clock cycle time and, 67
 - compiling C assignment with, 68
 - defined, 67
 - destination, 85, 274
 - floating-point, 226
 - left half, 301
 - LEGv8 conventions, 108
 - mapping, 82
 - number specification, 264
 - page table, 446
 - pipeline, 319, 321, 323
 - primitives, 67
 - renaming, 348
 - right half, 301
 - spilling, 72
 - Status, 337
 - temporary, 68, 102
 - variables, 67
 - Relative performance, 31–32
 - Relative power, 41
 - Reliability, 432
 - Remainder
 - defined, 198
 - Reorder buffers, 355
 - Replication, 479
 - Requested word first, 406
 - Request-level parallelism, 548
 - Reservation stations
 - buffering operands in, 350
 - defined, 350
 - Response time, 30–31
 - Restartable instructions, 462
 - Return address, 100
 - Return from exception (ERET), 459
 - R-format, 274
 - ALU operations, 265
 - defined, 86
 - Ripple carry
 - adder, A-29
 - carry lookahead speed versus, A-45–46
 - Roofline model, 558–559, 560, 561
 - with ceilings, 561
 - computational roofline, 559
 - illustrated, 557
 - Opteron generations, 558
 - with overlapping areas shaded, 563
 - peak floating-point performance, 562
 - peak memory performance, 562
 - with two kernels, 563
 - Rotational delay. *See* Rotational latency
 - Rotational latency, 397
 - Rounding, 226
 - accurate, 226
 - bits, 228
 - with guard digits, 227
 - IEEE 754 modes, 227
 - Row-major order, 225, 427
 - R-type instructions, 264
 - datapath for, 276
 - datapath in operation for, 278
- S**
- Saturation, 191
 - SCALAPAK, 244
 - Scaling
 - strong, 521
 - weak, 521
 - Scientific notation
 - adding numbers in, 213
 - defined, 205
 - for reals, 205
 - Search engines, 4
 - Secondary memory, 23
 - Sectors, 395
 - Secure Hash Algorithm (SHA)
 - encryption, 488
 - Seek, 396
 - Segmentation, 445
 - Selector values, A-10
 - Semiconductors, 25
 - Send message routine, 545
 - Sensitivity list, A-24
 - Sequencers
 - explicit, C-32
 - implementing next-state function with, C-22–28
 - Sequential logic, A-5
 - Servers, OL5.11-7. *See also* Desktop and server RISCs
 - cost and capability, 5
 - Service accomplishment, 432
 - Service interruption, 432
 - Set instructions, 97
 - Set-associative caches, 417. *See also* Caches
 - address portions, 421
 - block replacement strategies, 468
 - choice of, 467
 - four-way, 418, 421
 - memory-block location, 417
 - misses, 419–420
 - n*-way, 417
 - two-way, 418
 - Setup time, A-52, A-53
 - Shaders
 - defined, B-14
 - floating-point arithmetic, B-14
 - graphics, B-14–15
 - pixel example, B-15–17
 - Shading languages, B-14
 - Shadowing, OL5.11-5
 - Shared memory. *See also* Memory as low-latency memory, B-21
 - caching in, B-58–60
 - CUDA, B-58
 - N-body and, B-67–68
 - per-CTA, B-39
 - SRAM banks, B-40
 - Shared memory multiprocessors (SMP), 531–535
 - defined, 517, 531
 - single physical address space, 531
 - synchronization, 534
 - Shift amount, 84
 - Shift instructions, 90
 - Sign and magnitude, 206
 - Sign bit, 78
 - Sign extension, 266
 - defined, 78
 - shortcut, 80
 - Signals
 - asserted, 262, A-4
 - control, 262, 274–275
 - deasserted, 262, A-4
 - Signed division, 201–202
 - Signed multiplication, 196
 - Signed numbers, 75–82
 - sign and magnitude, 77
 - treating as unsigned, 98
 - Significands, 207
 - addition, 212
 - multiplication, 215
 - Silicon, 25
 - as key hardware technology, 53
 - crystal ingot, 26

- defined, 26
- wafers, 26
- Silicon crystal ingot, 26
- SIMD (Single Instruction Multiple Data), 522, 574
 - computers, OL6.15-2–6.15-4
 - data vector, B-35
 - extensions, OL6.15-4
 - for loops and, OL6.15-3
 - massively parallel multiprocessors, OL6.15-2
 - small-scale, OL6.15-4
 - vector architecture, 524–525
 - in x86, 524
- SIMMs (single inline memory modules), OL5.17-5, OL5.17-6
- Simple programmable logic devices (SPLDs), A-77
- Simplicity, 171
- Simultaneous multithreading (SMT), 531–533
 - support, 531
 - thread-level parallelism, 531
 - unused issue slots, 531
- Single error correcting/Double error correcting (SEC/DEC), 434–436
- Single instruction single data (SISD), 523
- Single precision. *See also* Double precision
 - binary representation, 210
 - defined, 207
- Single-clock-cycle pipeline diagrams, 308
 - illustrated, 310
- Single-cycle datapaths. *See also* Datapaths
 - illustrated, 298
 - instruction execution, 299
- Single-cycle implementation
 - control function for, 281
 - defined, 281
 - nonpipelined execution *versus* pipelined execution, 287
 - non-use of, 284
 - penalty, 283
 - pipelined performance *versus*, 285
- Single-instruction multiple-thread (SIMT), B-27–30
 - overhead, B-35
 - multithreaded warp scheduling, B-28
 - processor architecture, B-28
 - warp execution and divergence, B-29–30
- Single-program multiple data (SPMD), B-22
 - Smalltalk-80, OL2.22-8
 - Smart phones, 7
 - Snooping protocol, 479–481
 - Snoopy cache coherence, OL5.12-7
 - Software optimization
 - via blocking, 427–432
 - Sort algorithms, 146
 - Software
 - layers, 13
 - multiprocessor, 516
 - parallel, 517
 - as service, 7, 547, 574
 - systems, 13
 - Sort procedure, 140–144. *See also* Procedures
 - code for body, 140–142
 - full procedure, 143–144
 - passing parameters in, 143
 - preserving registers in, 143
 - procedure call, 143
 - register allocation for, 140
 - Sorting performance, B-54–55
 - Space allocation
 - on heap, 107–110
 - on stack, 106
 - SPARC
 - annulling branch, D-23
 - CASA, D-31
 - conditional branches, D-10–12
 - fast traps, D-30
 - floating-point operations, D-31
 - instructions, D-29–32
 - least significant bits, D-31
 - multiple precision floating-point results, D-32
 - nonfaulting loads, D-32
 - overlapping integer operations, D-31
 - quadruple precision floating-point arithmetic, D-32
 - register windows, D-29–30
 - support for LISP and Smalltalk, D-30
 - Sparse matrices, B-55–58
 - Sparse Matrix-Vector multiply (SpMV), B-55, B-57, B-58
 - CUDA version, B-57
 - serial code, B-57
 - shared memory version, B-59
 - Spatial locality, 388
 - large block exploitation of, 405
 - tendency, 392
 - SPEC, OL1.12-11–1.12-12
 - CPU benchmark, 46–48
 - power benchmark, 48–49
 - SPEC2000, OL1.12-12
 - SPEC2006, 246, OL1.12-12
 - SPEC89, OL1.12-11
 - SPEC92, OL1.12-12
 - SPEC95, OL1.12-12
 - SPECrate, 554
 - SPECratio, 47
 - Special function units (SFUs), B-35, B-50
 - defined, B-43
 - Speculation, 344–345
 - hardware-based, 352
 - implementation, 344
 - performance and, 344
 - problems, 344
 - recovery mechanism, 344
 - Speed-up challenge, 518
 - balancing load, 518–519
 - bigger problem, 520–521
 - Spilling registers, 72, 101
 - Split algorithm, 568
 - Split caches, 411
 - Stack architectures, OL2.22-4
 - Stack pointers
 - adjustment, 104
 - defined, 101
 - values, 104
 - Stacks
 - allocating space on, 106
 - for arguments, 145
 - defined, 101
 - pop, 101
 - push, 101, 104
 - Stalls, 291
 - as solution to control hazard, 292
 - avoiding with code reordering, 291
 - behavioral Verilog with detection, OL4.13-6
 - data hazards and, 324–328
 - illustrations, OL4.13-23, OL4.13-30
 - insertion into pipeline, 326
 - load-use, 329
 - memory, 414
 - write-back scheme, 413
 - write buffer, 413
 - Standby spares, OL5.11-8
 - State
 - in 2-bit prediction scheme, 333
 - assignment, A-69, C-27
 - bits, C-8
 - exception, saving/restoring, 462
 - logic components, 261
 - specification of, 446

- State elements
 - clock and, 262
 - combinational logic and, 262
 - defined, 260, A-47
 - inputs, 261
 - in storing/accessing instructions, 264
 - register file, A-49
 - Static branch prediction, 345
 - Static data
 - segment, 107
 - Static multiple-issue processors, 343, 345–349. *See also* Multiple issue
 - control hazards and, 345
 - instruction sets, 345
 - with LEGv8 ISA, 345–348
 - Static random access memories (SRAMs), 392, 393, A-57–61
 - array organization, A-61
 - basic structure, A-60
 - defined, 21, A-57
 - fixed access time, A-57
 - large, A-58
 - read/write initiation, A-58
 - synchronous (SSRAMs), A-59
 - three-state buffers, A-58, A-59
 - Static variables, 106
 - Steady-state prediction, 333
 - Sticky bits, 228
 - Store buffers, 355
 - Store instructions. *See also* Load
 - instructions
 - access, B-41
 - base register, 274
 - compiling with, 71
 - conditional, 126
 - defined, 71
 - EX stage, 305
 - ID stage, 302
 - IF stage, 302
 - instruction dependency, 323
 - MEM stage, 304
 - unit for implementing, 267
 - WB stage, 304
 - Store register, 72
 - Stored program concept, 63
 - as computer principle, 88
 - illustrated, 89
 - principles, 171
 - Strcpy procedure, 112–113. *See also* Procedures
 - as leaf procedure, 113
 - pointers, 113
 - Stream benchmark, 564
 - Streaming multiprocessor (SM), B-48–49
 - Streaming processors, B-34, B-49–50
 - array (SPA), B-41, B-46
 - Streaming SIMD Extension 2 (SSE2)
 - floating-point architecture, 232
 - Streaming SIMD Extensions (SSE) and
 - advanced vector extensions in x86, 232–233
 - Stretch computer, OL4.16-2
 - Strings
 - defined, 111
 - in Java, 113–115
 - representation, 111
 - Strip mining, 526
 - Striping, OL5.11-4
 - Strong scaling, 521
 - Structural hazards, 288–289, 305
 - STUR (store register), 64
 - STURB (store byte), 64
 - STURH (store half), 64
 - STURW (store word), 64
 - STXR (store exclusive register), 64, 126
 - SUB (subtract), 64
 - SUBI (subtract immediate), 64
 - SUBIS (subtract immediate and set flags), 64
 - SUBS (subtract and set flags), 64
 - Subnormals, 230
 - Subtraction, 188–191. *See also* Arithmetic
 - binary, 188–189
 - floating-point, 220
 - negative number, 190
 - overflow, 190
 - Subword parallelism, 230–232, 365, D-17
 - and matrix multiply, 238–242
 - Sum of products, A-11, A-12
 - Supercomputers, OL4.16-3
 - defined, 5
 - SuperH, D-15, D-39–40
 - Superscalars
 - defined, 349, OL4.16-5
 - dynamic pipeline scheduling, 349
 - multithreading options, 516
 - Surfaces, B-41
 - Swap procedure, 138. *See also* Procedures
 - body code, 138
 - full, 139–140, 143–145
 - register allocation, 138
 - Swap space, 448
 - Symbol tables, 130
 - Synchronization, 125–127, 568
 - barrier, B-18, B-20, B-34
 - defined, 534
 - lock, 125
 - overhead, reducing, 44–45
 - unlock, 125
 - Synchronizers
 - defined, A-75
 - failure, A-76
 - from D flip-flop, A-75
 - Synchronous DRAM (SRAM), 393–394, A-59, A-64
 - Synchronous SRAM (SSRAM), A-59
 - Synchronous system, A-47
 - Syntax tree, OL2.15-3
 - System calls
 - defined, 459
 - Systems software, 13
 - SystemVerilog
 - cache controller, OL5.12-2
 - cache data and tag modules, OL5.12-6
 - FSM, OL5.12-7
 - simple cache block diagram, OL5.12-4
 - type declarations, OL5.12-2
- ## T
- Tablets, 7
 - Tags
 - defined, 398
 - in locating block, 421
 - page tables and, 448
 - size of, 423
 - Tail call, 109
 - Task identifiers, 460
 - Task parallelism, B-24
 - Task-level parallelism, 516
 - Tebibyte (TiB), 5
 - Telsa PTX ISA, B-31–34
 - arithmetic instructions, B-33
 - barrier synchronization, B-34
 - GPU thread instructions, B-32
 - memory access instructions, B-33–34
 - Temporal locality, 388
 - tendency, 392
 - Temporary registers, 68, 102
 - Terabyte (TB), 6
 - defined, 5
 - Texture memory, B-40
 - Texture/processor cluster (TPC), B-47–48
 - TFLOPS multiprocessor, OL6.15-6

- Thrashing, 464
 - Thread blocks, 542
 - creation, B-23
 - defined, B-19
 - managing, B-30
 - memory sharing, B-20
 - synchronization, B-20
 - Thread parallelism, B-22
 - Threads
 - creation, B-23
 - CUDA, B-36
 - ISA, B-31–34
 - managing, B-30
 - memory latencies and, B-74–75
 - multiple, per body, B-68–69
 - warps, B-27
 - Three Cs model, 459–461
 - Three-state buffers, A-58, A-59
 - Throughput
 - defined, 30–31
 - multiple issue and, 342
 - pipelining and, 286, 342
 - Thumb, D-15, D-38
 - Timing
 - asynchronous inputs, A-75–76
 - level-sensitive, A-74–75
 - methodologies, A-71–76
 - two-phase, A-74
 - TLB misses, 453. *See also* Translation-lookaside buffer (TLB)
 - handling, 461–465
 - occurrence, 461
 - problem, 464
 - Tomasulo's algorithm, OL4.16-3
 - Touchscreen, 19
 - Tournament branch predictors, 334
 - Tracks, 395–396
 - Transfer time, 397
 - Transistors, 25
 - Translation Table Base Register (TTBR), 449
 - Translation-lookaside buffer (TLB), 452–454, D-26–27, OL5.17-6. *See also* TLB misses
 - associativities, 454
 - illustrated, 453
 - integration, 457
 - Intrinsity FastMATH, 454–457
 - typical values, 454
 - Transmit driver and NIC hardware
 - time *versus* receive driver and NIC hardware time, OL6.9-8
 - Tree-based parallel scan, B-62
 - Truth tables, A-5
 - ALU control lines, C-5
 - for control bits, 272
 - datapath control outputs, C-17
 - datapath control signals, C-14
 - defined, 272
 - example, A-5
 - next-state output bits, C-15
 - PLA implementation, A-13
 - Two's complement representation, 77–78
 - advantage, 78
 - negation shortcut, 79–80
 - rule, 81
 - sign extension shortcut, 80–81
 - Two-level logic, A-11–14
 - Two-phase clocking, A-74
 - TX-2 computer, OL6.15-4
- ## U
- Unconditional branches, 94
 - Underflow, 206
 - Unicode
 - alphabets, 113
 - defined, 113
 - example alphabets, 114
 - Unified GPU architecture, B-10–12
 - illustrated, B-11
 - processor array, B-11–12
 - Uniform memory access (UMA), 534, B-9
 - multiprocessors, 534
 - Units
 - commit, 350, 355
 - control, 259, 271–273, C-4–8, C-10, C-12–13
 - defined, 227
 - floating point, 227
 - hazard detection, 324, 327–328
 - for load/store implementation, 267
 - special function (SFUs), B-35, B-43, B-50
 - UNIVAC I, OL1.12-5
 - UNIX, OL2.22-8, OL5.17-9–5.17-12
 - AT&T, OL5.17-10
 - Berkeley version (BSD), OL5.17-10
 - genius, OL5.17-12
 - history, OL5.17-9–5.17-12
 - Unlock synchronization, 125
 - Unscaled signed immediate off set, 166
 - Unsigned numbers, 75–82
 - Use latency
 - defined, 346
 - one-instruction, 346
- ## V
- Vacuum tubes, 25
 - Valid bit, 400
 - Variables
 - C language, 106
 - programming language, 67
 - register, 67
 - static, 106
 - storage class, 106
 - type, 106
 - VAX architecture, OL2.22-4, OL5.17-7
 - Vector lanes, 526
 - Vector processors, 523–524. *See also* Processors
 - conventional code comparison, 525–526
 - instructions, 524
 - multimedia extensions and, 524–525
 - scalar *versus*, 526–527
 - Vectored interrupts, 337
 - Verilog
 - behavioral definition of MIPS ALU, A-25
 - behavioral definition with bypassing, OL4.13-4–4.13-6
 - behavioral definition with stalls for loads, OL4.13-6
 - behavioral specification, A-21, OL4.13-2–4.13-4
 - behavioral specification of multicycle MIPS design, OL4.13-12–4.13-13
 - behavioral specification with simulation, OL4.13-2
 - behavioral specification with stall detection, OL4.13-6
 - behavioral specification with synthesis, OL4.13-11–4.13-16
 - blocking assignment, A-24
 - branch hazard logic implementation, OL4.13-8–4.13-10
 - combinational logic, A-23–26
 - datatypes, A-21–22
 - defined, A-20
 - forwarding implementation, OL4.13-4
 - LEGv8 ALU definition in, A-34–37
 - modules, A-23
 - multicycle MIPS datapath, OL4.13-14

- Verilog (*Continued*)
 - nonblocking assignment, A-24
 - operators, A-22
 - program structure, A-23
 - reg, A-21–22
 - sensitivity list, A-24
 - sequential logic
 - specification, A-55–57
 - structural specification, A-21
 - wire, A-21–22
 - Vertical microcode, C-32
 - Very large-scale integrated (VLSI)
 - circuits, 25
 - Very Long Instruction Word (VLIW)
 - defined, 345
 - first generation computers, OL4.16–5
 - processors, 345
 - VHDL, A-20–21
 - Video graphics array (VGA) controllers,
 - B-3–4
 - Virtual addresses
 - causing page faults, 462
 - defined, 442
 - mapping from, 442–443
 - size, 444
 - Virtual machine monitors (VMMs)
 - defined, 438
 - implementing, 494
 - laissez-faire attitude, 494
 - page tables, 463
 - in performance improvement, 441
 - requirements, 440
 - Virtual machines (VMs), 438–441
 - benefits, 438
 - illusion, 463
 - instruction set architecture support, 441
 - performance improvement, 441
 - for protection improvement, 438
 - Virtual memory, 441–465. *See also* Pages
 - address translation, 443, 452–454
 - integration, 457–459
 - for large virtual addresses, 450–451
 - mechanism, 464
 - motivations, 427–442
 - page faults, 442, 448
 - protection implementation, 459–460
 - segmentation, 445
 - summary, 463
 - virtualization of, 463
 - writes, 452
 - Virtualizable hardware, 440
 - Virtually addressed caches, 458
 - Visual computing, B-3
 - Volatile memory, 22
- W**
- Wafers, 26
 - defects, 26
 - dies, 26–27
 - yield, 27
 - Warehouse Scale Computers (WSCs), 7,
 - 545–550, 574
 - Warps, 544, B-27
 - Weak scaling, 521
 - Wear levelling, 395
 - While loops, 95
 - Whirlwind, OL5.17–2
 - Wide area networks (WANs), 24. *See also* Networks
 - Wide immediate operands, 115–117
 - Words
 - accessing, 69
 - defined, 67
 - double, 158
 - load, 69, 71
 - quad, 158
 - store, 71
 - Working set, 464
 - World Wide Web, 4
 - Worst-case delay, 283
 - Write buffers
 - defined, 408
 - stalls, 413
 - write-back cache, 409
 - Write invalidate protocols, 479
 - Write serialization, 479
 - Write-back caches. *See also* Caches
 - advantages, 469
 - cache coherency protocol, OL5.12–5
 - complexity, 409
 - defined, 408, 469
 - stalls, 413
 - write buffers, 409
 - Write-back stage
 - control line, 313
 - load instruction, 303
 - store instruction, 305
 - Writes
 - complications, 408
 - expense, 464
 - handling, 407–409
 - memory hierarchy handling of, 469–470
 - schemes, 408
 - virtual memory, 451
 - write-back cache, 408, 409
 - write-through cache, 408, 409
 - Write-stall cycles, 414
 - Write-through caches. *See also* Caches
 - advantages, 469
 - defined, 407, 469
 - tag mismatch, 408
- X**
- x86, 154–162
 - Advanced Vector Extensions in, 232
 - brief history, OL2.22–6
 - conclusion, 162
 - data addressing modes, 157–158
 - evolution, 154–157
 - first address specifier encoding, 162
 - instruction encoding, 161–162
 - instruction formats, 161
 - instruction set growth, 170
 - instruction types, 160
 - integer operations, 158–160
 - registers, 157
 - SIMD in, 522
 - Streaming SIMD Extensions in, 232–233
 - typical instructions/functions, 161
 - typical operations, 161
 - Xerox Alto computer, OL1.12–8
 - XMM, 232
- Y**
- Yahoo! Cloud Serving Benchmark (YCSB), 556
 - Yield, 27
 - YMM, 232
- Z**
- Zettabyte, 6

B

A P P E N D I X

*Imagination is more
important than
knowledge.*

Albert Einstein

On Science, 1930s

Graphics and Computing GPUs

John Nickolls

Director of Architecture
NVIDIA

David Kirk

Chief Scientist
NVIDIA

B.1	Introduction	B-3
B.2	GPU System Architectures	B-7
B.3	Programming GPUs	B-12
B.4	Multithreaded Multiprocessor Architecture	B-25
B.5	Parallel Memory System	B-36
B.6	Floating-point Arithmetic	B-41
B.7	Real Stuff: The NVIDIA GeForce 8800	B-46
B.8	Real Stuff: Mapping Applications to GPUs	B-55
B.9	Fallacies and Pitfalls	B-72
B.10	Concluding Remarks	B-76
B.11	Historical Perspective and Further Reading	B-77

B.1 Introduction

This appendix focuses on the **GPU**—the ubiquitous **graphics processing unit** in every PC, laptop, desktop computer, and workstation. In its most basic form, the GPU generates 2D and 3D graphics, images, and video that enable Window-based operating systems, graphical user interfaces, video games, visual imaging applications, and video. The modern GPU that we describe here is a highly parallel, highly multithreaded multiprocessor optimized for **visual computing**. To provide real-time visual interaction with computed objects via graphics, images, and video, the GPU has a unified graphics and computing architecture that serves as both a programmable graphics processor and a scalable parallel computing platform. PCs and game consoles combine a GPU with a CPU to form **heterogeneous systems**.

A Brief History of GPU Evolution

Fifteen years ago, there was no such thing as a GPU. Graphics on a PC were performed by a *video graphics array* (VGA) controller. A VGA controller was simply a memory controller and display generator connected to some DRAM. In the 1990s, semiconductor technology advanced sufficiently that more functions could be added to the VGA controller. By 1997, VGA controllers were beginning to incorporate some *three-dimensional* (3D) acceleration functions, including

graphics processing unit (GPU) A processor optimized for 2D and 3D graphics, video, visual computing, and display.

visual computing A mix of graphics processing and computing that lets you visually interact with computed objects via graphics, images, and video.

heterogeneous system A system combining different processor types. A PC is a heterogeneous CPU–GPU system.

hardware for triangle setup and rasterization (dicing triangles into individual pixels) and texture mapping and shading (applying “decals” or patterns to pixels and blending colors).

In 2000, the single chip graphics processor incorporated almost every detail of the traditional high-end workstation graphics pipeline and, therefore, deserved a new name beyond VGA controller. The term GPU was coined to denote that the graphics device had become a processor.

Over time, GPUs became more programmable, as programmable processors replaced fixed-function dedicated logic while maintaining the basic 3D graphics pipeline organization. In addition, computations became more precise over time, progressing from indexed arithmetic, to integer and fixed point, to single-precision floating-point, and recently to double-precision floating-point. GPUs have become massively parallel programmable processors with hundreds of cores and thousands of threads.

Recently, processor instructions and memory hardware were added to support general purpose programming languages, and a programming environment was created to allow GPUs to be programmed using familiar languages, including C and C++. This innovation makes a GPU a fully general-purpose, programmable, manycore processor, albeit still with some special benefits and limitations.

GPU Graphics Trends

GPUs and their associated drivers implement the OpenGL and DirectX models of graphics processing. OpenGL is an open standard for 3D graphics programming available for most computers. DirectX is a series of Microsoft multimedia programming interfaces, including Direct3D for 3D graphics. Since these **application programming interfaces (APIs)** have well-defined behavior, it is possible to build effective hardware acceleration of the graphics processing functions defined by the APIs. This is one of the reasons (in addition to increasing device density) why new GPUs are being developed every 12 to 18 months that double the performance of the previous generation on existing applications.

Frequent doubling of GPU performance enables new applications that were not previously possible. The intersection of graphics processing and parallel computing invites a new paradigm for graphics, known as visual computing. It replaces large sections of the traditional sequential hardware graphics pipeline model with programmable elements for geometry, vertex, and pixel programs. Visual computing in a modern GPU combines graphics processing and parallel computing in novel ways that permit new graphics algorithms to be implemented, and opens the door to entirely new parallel processing applications on pervasive high-performance GPUs.

Heterogeneous System

Although the GPU is arguably the most parallel and most powerful processor in a typical PC, it is certainly not the only processor. The CPU, now multicore and

application programming interface (API) A set of function and data structure definitions providing an interface to a library of functions.

soon to be manycore, is a complementary, primarily serial processor companion to the massively parallel manycore GPU. Together, these two types of processors comprise a heterogeneous multiprocessor system.

The best performance for many applications comes from using both the CPU and the GPU. This appendix will help you understand how and when to best split the work between these two increasingly parallel processors.

GPU Evolves into Scalable Parallel Processor

GPUs have evolved functionally from hardwired, limited capability VGA controllers to programmable parallel processors. This evolution has proceeded by changing the logical (API-based) graphics pipeline to incorporate programmable elements and also by making the underlying hardware pipeline stages less specialized and more programmable. Eventually, it made sense to merge disparate programmable pipeline elements into one unified array of many programmable processors.

In the GeForce 8-series generation of GPUs, the geometry, vertex, and pixel processing all run on the same type of processor. This unification allows for dramatic scalability. More programmable processor cores increase the total system throughput. Unifying the processors also delivers very effective load balancing, since any processing function can use the whole processor array. At the other end of the spectrum, a processor array can now be built with very few processors, since all of the functions can be run on the same processors.

Why CUDA and GPU Computing?

This uniform and scalable array of processors invites a new model of programming for the GPU. The large amount of floating-point processing power in the GPU processor array is very attractive for solving nongraphics problems. Given the large degree of parallelism and the range of scalability of the processor array for graphics applications, the programming model for more general computing must express the massive parallelism directly, but allow for scalable execution.

GPU computing is the term coined for using the GPU for computing via a parallel programming language and API, without using the traditional graphics API and graphics pipeline model. This is in contrast to the earlier **General Purpose computation on GPU (GPGPU)** approach, which involves programming the GPU using a graphics API and graphics pipeline to perform nongraphics tasks.

Compute Unified Device Architecture (CUDA) is a scalable parallel programming model and software platform for the GPU and other parallel processors that allows the programmer to bypass the graphics API and graphics interfaces of the GPU and simply program in C or C++. The CUDA programming model has an SPMD (single-program multiple data) software style, in which a programmer writes a program for one thread that is instanced and executed by many threads in parallel on the multiple processors of the GPU. In fact, CUDA also provides a facility for programming multiple CPU cores as well, so CUDA is an environment for writing parallel programs for the entire heterogeneous computer system.

GPU computing Using a GPU for computing via a parallel programming language and API.

GPGPU Using a GPU for general-purpose computation via a traditional graphics API and graphics pipeline.

CUDA A scalable parallel programming model and language based on C/C++. It is a parallel programming platform for GPUs and multicore CPUs.

GPU Unifies Graphics and Computing

With the addition of CUDA and GPU computing to the capabilities of the GPU, it is now possible to use the GPU as both a graphics processor and a computing processor at the same time, and to combine these uses in visual computing applications. The underlying processor architecture of the GPU is exposed in two ways: first, as implementing the programmable graphics APIs, and second, as a massively parallel processor array programmable in C/C++ with CUDA.

Although the underlying processors of the GPU are unified, it is not necessary that all of the SPMD thread programs are the same. The GPU can run graphics shader programs for the graphics aspect of the GPU, processing geometry, vertices, and pixels, and also run thread programs in CUDA.

The GPU is truly a versatile multiprocessor architecture, supporting a variety of processing tasks. GPUs are excellent at graphics and visual computing as they were specifically designed for these applications. GPUs are also excellent at many general-purpose throughput applications that are “first cousins” of graphics, in that they perform a lot of parallel work, as well as having a lot of regular problem structure. In general, they are a good match to data-parallel problems (see [Chapter 6](#)), particularly large problems, but less so for less regular, smaller problems.

GPU Visual Computing Applications

Visual computing includes the traditional types of graphics applications plus many new applications. The original purview of a GPU was “anything with pixels,” but it now includes many problems without pixels but with regular computation and/or data structure. GPUs are effective at 2D and 3D graphics, since that is the purpose for which they are designed. Failure to deliver this application performance would be fatal. 2D and 3D graphics use the GPU in its “graphics mode,” accessing the processing power of the GPU through the graphics APIs, OpenGL™, and DirectX™. Games are built on the 3D graphics processing capability.

Beyond 2D and 3D graphics, image processing and video are important applications for GPUs. These can be implemented using the graphics APIs or as computational programs, using CUDA to program the GPU in computing mode. Using CUDA, image processing is simply another data-parallel array program. To the extent that the data access is regular and there is good locality, the program will be efficient. In practice, image processing is a very good application for GPUs. Video processing, especially encode and decode (compression and decompression according to some standard algorithms), is quite efficient.

The greatest opportunity for visual computing applications on GPUs is to “break the graphics pipeline.” Early GPUs implemented only specific graphics APIs, albeit at very high performance. This was wonderful if the API supported the operations that you wanted to do. If not, the GPU could not accelerate your task, because early GPU functionality was immutable. Now, with the advent of GPU computing and CUDA, these GPUs can be programmed to implement a different virtual pipeline by simply writing a CUDA program to describe the computation and data flow that is desired. So, all applications are now possible, which will stimulate new visual computing approaches.

B.2 GPU System Architectures

In this section, we survey GPU system architectures in common use today. We discuss system configurations, GPU functions and services, standard programming interfaces, and a basic GPU internal architecture.

Heterogeneous CPU–GPU System Architecture

A heterogeneous computer system architecture using a GPU and a CPU can be described at a high level by two primary characteristics: first, how many functional subsystems and/or chips are used and what are their interconnection technologies and topology; and second, what memory subsystems are available to these functional subsystems. See [Chapter 6](#) for background on the PC I/O systems and chip sets.

The Historical PC (circa 1990)

[Figure B.2.1](#) shows a high-level block diagram of a legacy PC, circa 1990. The north bridge (see [Chapter 6](#)) contains high-bandwidth interfaces, connecting the CPU, memory, and PCI bus. The south bridge contains legacy interfaces and devices: ISA bus (audio, LAN), interrupt controller; DMA controller; time/counter. In this system, the display was driven by a simple framebuffer subsystem known

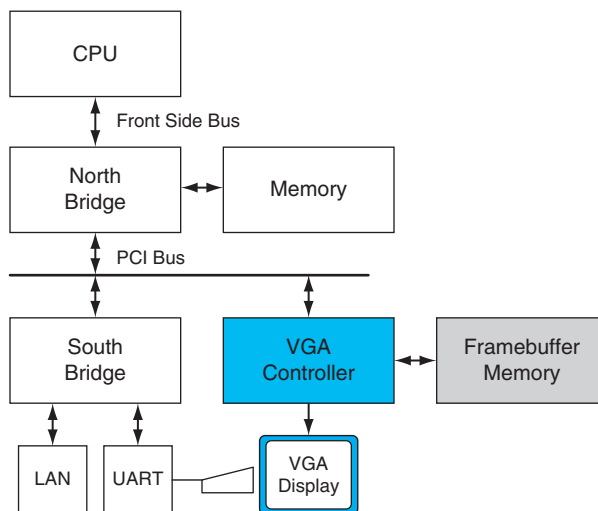


FIGURE B.2.1 Historical PC. VGA controller drives graphics display from framebuffer memory.

PCI-Express (PCIe)

A standard system I/O interconnect that uses point-to-point links. Links have a configurable number of lanes and bandwidth.

as a VGA (*video graphics array*) which was attached to the PCI bus. Graphics subsystems with built-in processing elements (GPUs) did not exist in the PC landscape of 1990.

Figure B.2.2 illustrates two configurations in common use today. These are characterized by a separate GPU (discrete GPU) and CPU with respective memory subsystems. In Figure B.2.2a, with an Intel CPU, we see the GPU attached via a 16-lane **PCI-Express** 2.0 link to provide a peak 16GB/s transfer rate (peak of 8 GB/s in each direction). Similarly, in Figure B.2.2b, with an AMD CPU, the GPU

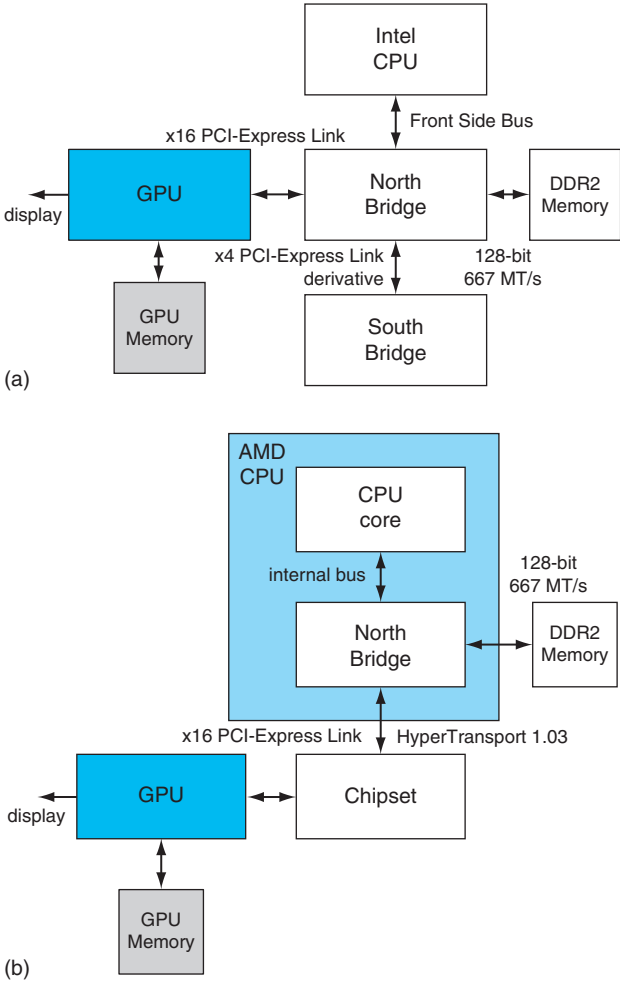


FIGURE B.2.2 Contemporary PCs with Intel and AMD CPUs. See Chapter 6 for an explanation of the components and interconnects in this figure.

is attached to the chipset, also via PCI-Express with the same available bandwidth. In both cases, the GPUs and CPUs may access each other's memory, albeit with less available bandwidth than their access to the more directly attached memories. In the case of the AMD system, the north bridge or memory controller is integrated into the same die as the CPU.

A low-cost variation on these systems, a **unified memory architecture (UMA)** system, uses only CPU system memory, omitting GPU memory from the system. These systems have relatively low-performance GPUs, since their achieved performance is limited by the available system memory bandwidth and increased latency of memory access, whereas dedicated GPU memory provides high bandwidth and low latency.

A high-performance system variation uses multiple attached GPUs, typically two to four working in parallel, with their displays daisy-chained. An example is the NVIDIA SLI (scalable link interconnect) multi-GPU system, designed for high-performance gaming and workstations.

The next system category integrates the GPU with the north bridge (Intel) or chipset (AMD) with and without dedicated graphics memory.

Chapter 5 explains how caches maintain coherence in a shared address space. With CPUs and GPUs, there are multiple address spaces. GPUs can access their own physical local memory and the CPU system's physical memory using virtual addresses that are translated by an MMU on the GPU. The operating system kernel manages the GPU's page tables. A system physical page can be accessed using either coherent or noncoherent PCI-Express transactions, determined by an attribute in the GPU's page table. The CPU can access GPU's local memory through an address range (also called aperture) in the PCI-Express address space.

Game Consoles

Console systems such as the Sony PlayStation 3 and the Microsoft Xbox 360 resemble the PC system architectures previously described. Console systems are designed to be shipped with identical performance and functionality over a lifespan that can last five years or more. During this time, a system may be reimplemented many times to exploit more advanced silicon manufacturing processes and thereby to provide constant capability at ever lower costs. Console systems do not need to have their subsystems expanded and upgraded the way PC systems do, so the major internal system buses tend to be customized rather than standardized.

GPU Interfaces and Drivers

In a PC today, GPUs are attached to a CPU via PCI-Express. Earlier generations used **AGP**. Graphics applications call OpenGL [Segal and Akeley, 2006] or DirectX3D [Microsoft DirectX Specification] API functions that use the GPU as a coprocessor. The APIs send commands, programs, and data to the GPU via a graphics device driver optimized for the particular GPU.

unified memory architecture (UMA)

A system architecture in which the CPU and GPU share a common system memory.

AGP An extended version of the original PCI I/O bus, which provided up to eight times the bandwidth of the original PCI bus to a single card slot. Its primary purpose was to connect graphics subsystems into PC systems.

Graphics Logical Pipeline

The graphics logical pipeline is described in Section B.3. [Figure B.2.3](#) illustrates the major processing stages, and highlights the important programmable stages (vertex, geometry, and pixel shader stages).



FIGURE B.2.3 Graphics logical pipeline. Programmable graphics shader stages are blue, and fixed-function blocks are white.

Mapping Graphics Pipeline to Unified GPU Processors

[Figure B.2.4](#) shows how the logical pipeline comprising separate independent programmable stages is mapped onto a physical distributed array of processors.

Basic Unified GPU Architecture

Unified GPU architectures are based on a parallel array of many programmable processors. They unify vertex, geometry, and pixel shader processing and parallel computing on the same processors, unlike earlier GPUs which had separate processors dedicated to each processing type. The programmable processor array is tightly integrated with fixed function processors for texture filtering, rasterization, raster operations, anti-aliasing, compression, decompression, display, video decoding, and high-definition video processing. Although the fixed-function processors significantly outperform more general programmable processors in terms of absolute performance constrained by an area, cost, or power budget, we will focus on the programmable processors here.

Compared with multicore CPUs, manycore GPUs have a different architectural design point, one focused on executing many parallel threads efficiently on many

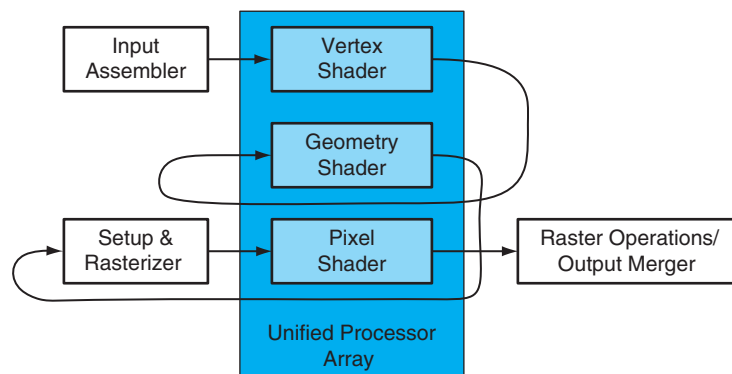


FIGURE B.2.4 Logical pipeline mapped to physical processors. The programmable shader stages execute on the array of unified processors, and the logical graphics pipeline dataflow recirculates through the processors.

processor cores. By using many simpler cores and optimizing for data-parallel behavior among groups of threads, more of the per-chip transistor budget is devoted to computation, and less to on-chip caches and overhead.

Processor Array

A unified GPU processor array contains many processor cores, typically organized into multithreaded multiprocessors. Figure B.2.5 shows a GPU with an array of 112 *streaming processor* (SP) cores, organized as 14 multithreaded *streaming multiprocessors* (SMs). Each SP core is highly multithreaded, managing 96 concurrent threads and their state in hardware. The processors connect with four 64-bit-wide DRAM partitions via an interconnection network. Each SM has eight SP cores, two *special function units* (SFUs), instruction and constant caches, a multithreaded instruction unit, and a shared memory. This is the basic Tesla architecture implemented by the NVIDIA GeForce 8800. It has a unified architecture in which the traditional graphics programs for vertex, geometry, and pixel shading run on the unified SMs and their SP cores, and computing programs run on the same processors.

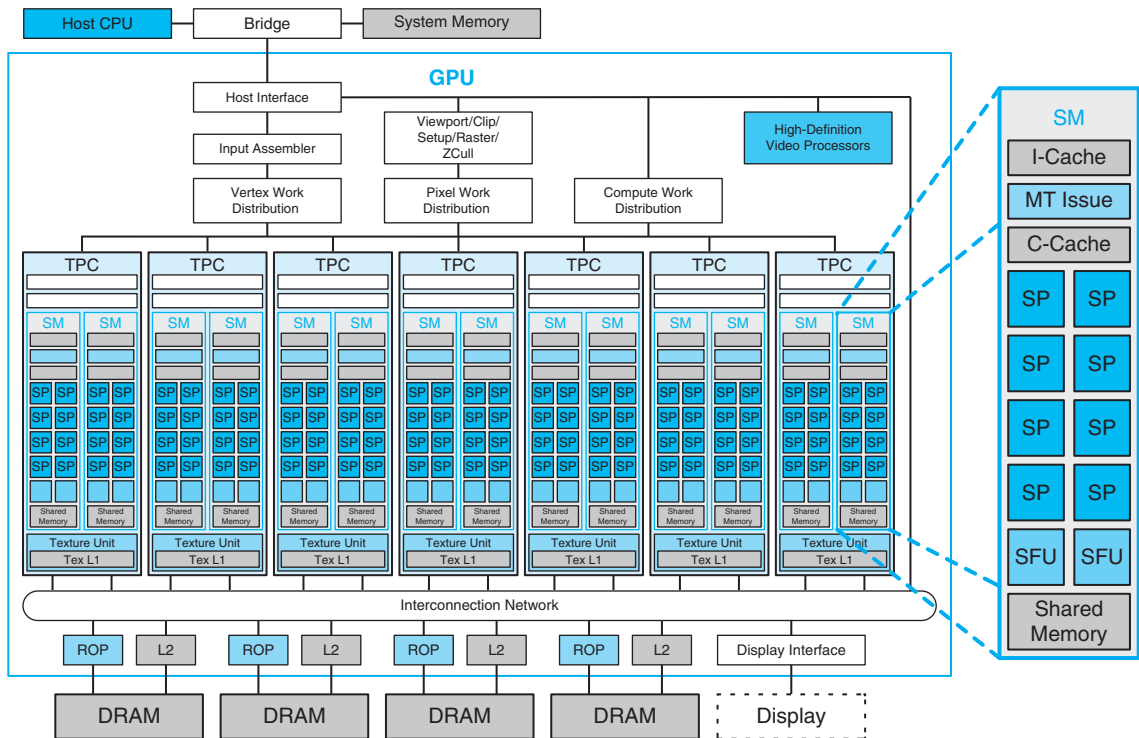


FIGURE B.2.5 Basic unified GPU architecture. Example GPU with 112 *streaming processor* (SP) cores organized in 14 *streaming multiprocessors* (SMs); the cores are highly multithreaded. It has the basic Tesla architecture of an NVIDIA GeForce 8800. The processors connect with four 64-bit-wide DRAM partitions via an interconnection network. Each SM has eight SP cores, two *special function units* (SFUs), instruction and constant caches, a multithreaded instruction unit, and a shared memory.

The processor array architecture is scalable to smaller and larger GPU configurations by scaling the number of multiprocessors and the number of memory partitions. Figure B.2.5 shows seven clusters of two SMs sharing a texture unit and a texture L1 cache. The texture unit delivers filtered results to the SM given a set of coordinates into a texture map. Because filter regions of support often overlap for successive texture requests, a small streaming L1 texture cache is effective to reduce the number of requests to the memory system. The processor array connects with *raster operation processors* (ROPs), L2 texture caches, external DRAM memories, and system memory via a GPU-wide interconnection network. The number of processors and number of memories can scale to design balanced GPU systems for different performance and market segments.

B.3 Programming GPUs

Programming multiprocessor GPUs is qualitatively different than programming other multiprocessors like multicore CPUs. GPUs provide two to three orders of magnitude more thread and data parallelism than CPUs, scaling to hundreds of processor cores and tens of thousands of concurrent threads. GPUs continue to increase their parallelism, doubling it about every 12 to 18 months, enabled by Moore's law [1965] of increasing integrated circuit density and by improving architectural efficiency. To span the wide price and performance range of different market segments, different GPU products implement widely varying numbers of processors and threads. Yet users expect games, graphics, imaging, and computing applications to work on any GPU, regardless of how many parallel threads it executes or how many parallel processor cores it has, and they expect more expensive GPUs (with more threads and cores) to run applications faster. As a result, GPU programming models and application programs are designed to scale transparently to a wide range of parallelism.

The driving force behind the large number of parallel threads and cores in a GPU is real-time graphics performance—the need to render complex 3D scenes with high resolution at interactive frame rates, at least 60 frames per second. Correspondingly, the scalable programming models of graphics shading languages such as Cg (C for graphics) and HLSL (*high-level shading language*) are designed to exploit large degrees of parallelism via many independent parallel threads and to scale to any number of processor cores. The CUDA scalable parallel programming model similarly enables general parallel computing applications to leverage large numbers of parallel threads and scale to any number of parallel processor cores, transparently to the application.

In these scalable programming models, the programmer writes code for a single thread, and the GPU runs myriad thread instances in parallel. Programs thus scale transparently over a wide range of hardware parallelism. This simple paradigm arose from graphics APIs and shading languages that describe how to shade one

vertex or one pixel. It has remained an effective paradigm as GPUs have rapidly increased their parallelism and performance since the late 1990s.

This section briefly describes programming GPUs for real-time graphics applications using graphics APIs and programming languages. It then describes programming GPUs for visual computing and general parallel computing applications using the C language and the CUDA programming model.

Programming Real-Time Graphics

APIs have played an important role in the rapid, successful development of GPUs and processors. There are two primary standard graphics APIs: **OpenGL** and **Direct3D**, one of the Microsoft DirectX multimedia programming interfaces. OpenGL, an open standard, was originally proposed and defined by Silicon Graphics Incorporated. The ongoing development and extension of the OpenGL standard [Segal and Akeley, 2006; Kessenich, 2006] is managed by Khronos, an industry consortium. Direct3D [Blythe, 2006], a de facto standard, is defined and evolved forward by Microsoft and partners. OpenGL and Direct3D are similarly structured, and continue to evolve rapidly with GPU hardware advances. They define a logical graphics processing pipeline that is mapped onto the GPU hardware and processors, along with programming models and languages for the programmable pipeline stages.

OpenGL An open-standard graphics API.

Direct3D A graphics API defined by Microsoft and partners.

Logical Graphics Pipeline

Figure B.3.1 illustrates the Direct3D 10 logical graphics pipeline. OpenGL has a similar graphics pipeline structure. The API and logical pipeline provide a streaming dataflow infrastructure and plumbing for the programmable shader stages, shown in blue. The 3D application sends the GPU a sequence of vertices grouped into geometric primitives—points, lines, triangles, and polygons. The input assembler collects vertices and primitives. The vertex shader program executes per-vertex processing,

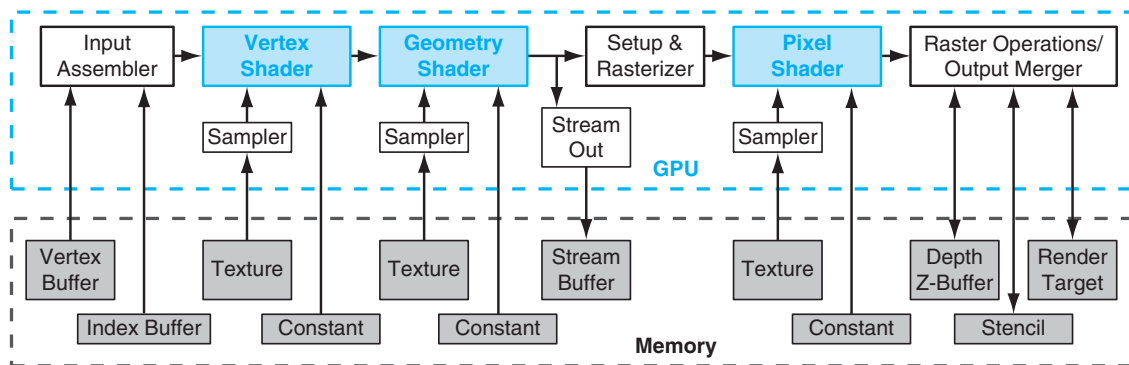


FIGURE B.3.1 Direct3D 10 graphics pipeline. Each logical pipeline stage maps to GPU hardware or to a GPU processor. Programmable shader stages are blue, fixed-function blocks are gray, and memory objects are gray. Each stage processes a vertex, geometric primitive, or pixel in a streaming dataflow fashion.

texture A 1D, 2D, or 3D array that supports sampled and filtered lookups with interpolated coordinates.

including transforming the vertex 3D position into a screen position and lighting the vertex to determine its color. The geometry shader program executes per-primitive processing and can add or drop primitives. The setup and rasterizer unit generates pixel fragments (fragments are potential contributions to pixels) that are covered by a geometric primitive. The pixel shader program performs per-fragment processing, including interpolating per-fragment parameters, texturing, and coloring. Pixel shaders make extensive use of sampled and filtered lookups into large 1D, 2D, or 3D arrays called **textures**, using interpolated floating-point coordinates. Shaders use texture accesses for maps, functions, decals, images, and data. The raster operations processing (or output merger) stage performs Z-buffer depth testing and stencil testing, which may discard a hidden pixel fragment or replace the pixel's depth with the fragment's depth, and performs a color blending operation that combines the fragment color with the pixel color and writes the pixel with the blended color.

The graphics API and graphics pipeline provide input, output, memory objects, and infrastructure for the shader programs that process each vertex, primitive, and pixel fragment.

Graphics Shader Programs

shader A program that operates on graphics data such as a vertex or a pixel fragment.

shading language
A graphics rendering language, usually having a dataflow or streaming programming model.

Real-time graphics applications use many different **shader** programs to model how light interacts with different materials and to render complex lighting and shadows. **Shading languages** are based on a dataflow or streaming programming model that corresponds with the logical graphics pipeline. Vertex shader programs map the position of triangle vertices onto the screen, altering their position, color, or orientation. Typically a vertex shader thread inputs a floating-point (x, y, z, w) vertex position and computes a floating-point (x, y, z) screen position. Geometry shader programs operate on geometric primitives (such as lines and triangles) defined by multiple vertices, changing them or generating additional primitives. Pixel fragment shaders each “shade” one pixel, computing a floating-point *red*, *green*, *blue*, *alpha* (RGBA) color contribution to the rendered image at its pixel sample (x, y) image position. Shaders (and GPUs) use floating-point arithmetic for all pixel color calculations to eliminate visible artifacts while computing the extreme range of pixel contribution values encountered while rendering scenes with complex lighting, shadows, and high dynamic range. For all three types of graphics shaders, many program instances can be run in parallel, as independent parallel threads, because each works on independent data, produces independent results, and has no side effects. Independent vertices, primitives, and pixels further enable the same graphics program to run on differently sized GPUs that process different numbers of vertices, primitives, and pixels in parallel. Graphics programs thus scale transparently to GPUs with different amounts of parallelism and performance.

Users program all three logical graphics threads with a common targeted high-level language. HLSL (high-level shading language) and Cg (C for graphics) are commonly used. They have C-like syntax and a rich set of library functions for matrix operations, trigonometry, interpolation, and texture access and filtering, but are far from general computing languages: they currently lack general memory

access, pointers, file I/O, and recursion. HLSL and Cg assume that programs live within a logical graphics pipeline, and thus I/O is implicit. For example, a pixel fragment shader may expect the geometric normal and multiple texture coordinates to have been interpolated from vertex values by upstream fixed-function stages and can simply assign a value to the COLOR output parameter to pass it downstream to be blended with a pixel at an implied (x, y) position.

The GPU hardware creates a new independent thread to execute a vertex, geometry, or pixel shader program for every vertex, every primitive, and every pixel fragment. In video games, the bulk of threads execute pixel shader programs, as there are typically 10 to 20 times more pixel fragments than vertices, and complex lighting and shadows require even larger ratios of pixel to vertex shader threads. The graphics shader programming model drove the GPU architecture to efficiently execute thousands of independent fine-grained threads on many parallel processor cores.

Pixel Shader Example

Consider the following Cg pixel shader program that implements the “environment mapping” rendering technique. For each pixel thread, this shader is passed five parameters, including 2D floating-point texture image coordinates needed to sample the surface color, and a 3D floating-point vector giving the refraction of the view direction off the surface. The other three “uniform” parameters do not vary from one pixel instance (thread) to the next. The shader looks up color in two texture images: a 2D texture access for the surface color, and a 3D texture access into a cube map (six images corresponding to the faces of a cube) to obtain the external world color corresponding to the refraction direction. Then the final four-component (red, green, blue, alpha) floating-point color is computed using a weighted average called a “lerp” or linear interpolation function.

```
void refraction(
    float2          texCoord      : TEXCOORD0,
    float3          refraction_dir : TEXCOORD1,
    out float4      color         : COLOR,
    uniform float    shiny,
    uniform sampler2D surfaceMap,
    uniform samplerCUBE envMap)
{
    // Fetch the surface color from a texture
    float4 surfaceColor = tex2D(surfaceMap, texCoord);

    // Fetch reflected color by sampling a cube map
    float4 reflectedColor = texCUBE(envMap, refraction_dir);

    // Output is weighted average of the two colors
    color = lerp(surfaceColor, reflectedColor, shiny);
}
```

Although this shader program is only three lines long, it activates a lot of GPU hardware. For each texture fetch, the GPU texture subsystem makes multiple memory accesses to sample image colors in the vicinity of the sampling coordinates, and then interpolates the final result with floating-point filtering arithmetic. The multithreaded GPU executes thousands of these lightweight Cg pixel shader threads in parallel, deeply interleaving them to hide texture fetch and memory latency.

Cg focuses the programmer's view to a single vertex or primitive or pixel, which the GPU implements as a single thread; the shader program transparently scales to exploit thread parallelism on the available processors. Being application-specific, Cg provides a rich set of useful data types, library functions, and language constructs to express diverse rendering techniques.

Figure B.3.2 shows skin rendered by a fragment pixel shader. Real skin appears quite different from flesh-color paint because light bounces around a lot before re-emerging. In this complex shader, three separate skin layers, each with unique subsurface scattering behavior, are modeled to give the skin a visual depth and translucency. Scattering can be modeled by a blurring convolution in a fattened “texture” space, with red being blurred more than green, and blue blurred less. The compiled Cg shader executes 1400 instructions to compute the color of one skin pixel.



FIGURE B.3.2 GPU-rendered image. To give the skin visual depth and translucency, the pixel shader program models three separate skin layers, each with unique subsurface scattering behavior. It executes 1400 instructions to render the red, green, blue, and alpha color components of each skin pixel fragment.

As GPUs have evolved superior floating-point performance and very high streaming memory bandwidth for real-time graphics, they have attracted highly parallel applications beyond traditional graphics. At first, access to this power was available only by couching an application as a graphics-rendering algorithm, but this GPGPU approach was often awkward and limiting. More recently, the CUDA programming model has provided a far easier way to exploit the scalable high-performance floating-point and memory bandwidth of GPUs with the C programming language.

Programming Parallel Computing Applications

CUDA, Brook, and CAL are programming interfaces for GPUs that are focused on data parallel computation rather than on graphics. CAL (*Compute Abstraction Layer*) is a low-level assembler language interface for AMD GPUs. Brook is a streaming language adapted for GPUs by [Buck et al. \[2004\]](#). CUDA, developed by [NVIDIA \[2007\]](#), is an extension to the C and C++ languages for scalable parallel programming of manycore GPUs and multicore CPUs. The CUDA programming model is described below, adapted from an article by [Nickolls et al. \[2008\]](#).

With the new model the GPU excels in data parallel and throughput computing, executing high-performance computing applications as well as graphics applications.

Data Parallel Problem Decomposition

To map large computing problems effectively to a highly parallel processing architecture, the programmer or compiler decomposes the problem into many small problems that can be solved in parallel. For example, the programmer partitions a large result data array into blocks and further partitions each block into elements, such that the result blocks can be computed independently in parallel, and the elements within each block are computed in parallel. [Figure B.3.3](#) shows a decomposition of a result data array into a 3×2 grid of blocks, where each block is further decomposed into a 5×3 array of elements. The two-level parallel decomposition maps naturally to the GPU architecture: parallel multiprocessors compute result blocks, and parallel threads compute result elements.

The programmer writes a program that computes a sequence of result data grids, partitioning each result grid into coarse-grained result blocks that can be computed independently in parallel. The program computes each result block with an array of fine-grained parallel threads, partitioning the work among threads so that each computes one or more result elements.

Scalable Parallel Programming with CUDA

The CUDA scalable parallel programming model extends the C and C++ languages to exploit large degrees of parallelism for general applications on highly parallel multiprocessors, particularly GPUs. Early experience with CUDA shows that *many* sophisticated programs can be readily expressed with a few easily understood abstractions. Since NVIDIA released CUDA in 2007, developers have

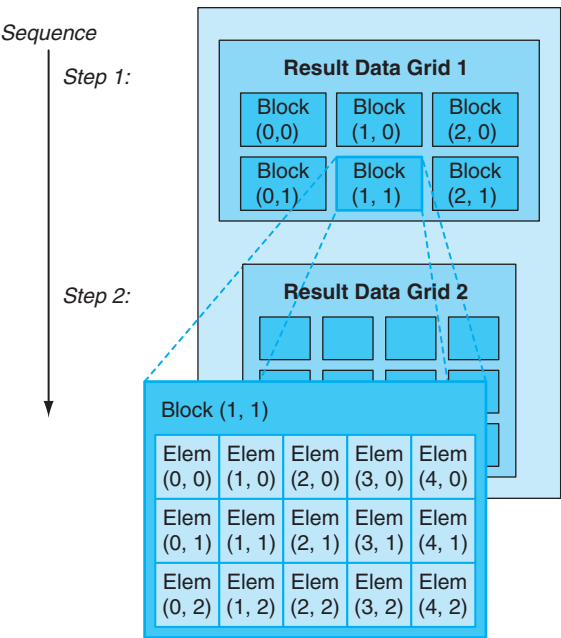


FIGURE B.3.3 Decomposing result data into a grid of blocks of elements to be computed in parallel.

rapidly developed scalable parallel programs for a wide range of applications, including seismic data processing, computational chemistry, linear algebra, sparse matrix solvers, sorting, searching, physics models, and visual computing. These applications scale transparently to hundreds of processor cores and thousands of concurrent threads. NVIDIA GPUs with the Tesla unified graphics and computing architecture (described in Sections B.4 and B.7) run CUDA C programs, and are widely available in laptops, PCs, workstations, and servers. The CUDA model is also applicable to other shared memory parallel processing architectures, including multicore CPUs.

CUDA provides three key abstractions—a *hierarchy of thread groups*, *shared memories*, and *barrier synchronization*—that provide a clear parallel structure to conventional C code for one thread of the hierarchy. Multiple levels of threads, memory, and synchronization provide fine-grained data parallelism and thread parallelism, nested within coarse-grained data parallelism and task parallelism. The abstractions guide the programmer to partition the problem into coarse subproblems that can be solved independently in parallel, and then into finer pieces that can be solved in parallel. The programming model scales transparently to large numbers of processor cores: a compiled CUDA program executes on any number of processors, and only the runtime system needs to know the physical processor count.

The CUDA Paradigm

CUDA is a minimal extension of the C and C++ programming languages. The programmer writes a serial program that calls parallel **kernels**, which may be simple functions or full programs. A kernel executes in parallel across a set of parallel threads. The programmer organizes these threads into a hierarchy of thread blocks and grids of thread blocks. A **thread block** is a set of concurrent threads that can cooperate among themselves through barrier synchronization and through shared access to a memory space private to the block. A **grid** is a set of thread blocks that may each be executed independently and thus may execute in parallel.

When invoking a kernel, the programmer specifies the number of threads per block and the number of blocks comprising the grid. Each thread is given a unique *thread ID* number `threadIdx` within its thread block, numbered `0`, `1`, `2`, ..., `blockDim-1`, and each thread block is given a unique *block ID* number `blockIdx` within its grid. CUDA supports thread blocks containing up to 512 threads. For convenience, thread blocks and grids may have one, two, or three dimensions, accessed via `.x`, `.y`, and `.z` index fields.

As a very simple example of parallel programming, suppose that we are given two vectors x and y of n floating-point numbers each and that we wish to compute the result of $y = ax + y$ for some scalar value a . This is the so-called SAXPY kernel defined by the BLAS linear algebra library. Figure B.3.4 shows C code for performing this computation on both a serial processor and in parallel using CUDA.

The `__global__` declaration specifier indicates that the procedure is a kernel entry point. CUDA programs launch parallel kernels with the extended function call syntax:

```
kernel<<<dimGrid, dimBlock>>>(... parameter list ...);
```

where `dimGrid` and `dimBlock` are three-element vectors of type `dim3` that specify the dimensions of the grid in blocks and the dimensions of the blocks in threads, respectively. Unspecified dimensions default to one.

In Figure B.3.4, we launch a grid of n threads that assigns one thread to each element of the vectors and puts 256 threads in each block. Each individual thread computes an element index from its thread and block IDs and then performs the desired calculation on the corresponding vector elements. Comparing the serial and parallel versions of this code, we see that they are strikingly similar. This represents a fairly common pattern. The serial code consists of a loop where each iteration is independent of all the others. Such loops can be mechanically transformed into parallel kernels: each loop iteration becomes an independent thread. By assigning a single thread to each output element, we avoid the need for any synchronization among threads when writing results to memory.

The text of a CUDA kernel is simply a C function for one sequential thread. Thus, it is generally straightforward to write and is typically simpler than writing parallel code for vector operations. Parallelism is determined clearly and explicitly by specifying the dimensions of a grid and its thread blocks when launching a kernel.

kernel A program or function for one thread, designed to be executed by many threads.

thread block A set of concurrent threads that execute the same thread program and may cooperate to compute a result.

grid A set of thread blocks that execute the same kernel program.

Computing $y = ax + y$ with a serial loop:

```

void saxpy_serial(int n, float alpha, float *x, float *y)
{
    for(int i = 0; i<n; ++i)
        y[i] = alpha*x[i] + y[i];
}
// Invoke serial SAXPY kernel
saxpy_serial(n, 2.0, x, y);

```

Computing $y = ax + y$ in parallel using CUDA:

```

__global__
void saxpy_parallel(int n, float alpha, float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;

    if( i<n ) y[i] = alpha*x[i] + y[i];
}

// Invoke parallel SAXPY kernel (256 threads per block)
int nblocks = (n + 255) / 256;
saxpy_parallel<<<nblocks, 256>>>(n, 2.0, x, y);

```

FIGURE B.3.4 Sequential code (top) in C versus parallel code (bottom) in CUDA for SAXPY (see Chapter 6). CUDA parallel threads replace the C serial loop—each thread computes the same result as one loop iteration. The parallel code computes n results with n threads organized in blocks of 256 threads.

synchronization

barrier Threads wait at a synchronization barrier until all threads in the thread block arrive at the barrier.

Parallel execution and thread management is automatic. All thread creation, scheduling, and termination is handled for the programmer by the underlying system. Indeed, a Tesla architecture GPU performs all thread management directly in hardware. The threads of a block execute concurrently and may synchronize at a **synchronization barrier** by calling the `__syncthreads()` intrinsic. This guarantees that no thread in the block can proceed until all threads in the block have reached the barrier. After passing the barrier, these threads are also guaranteed to see all writes to memory performed by threads in the block before the barrier. Thus, threads in a block may communicate with each other by writing and reading per-block shared memory at a synchronization barrier.

Since threads in a block may share memory and synchronize via barriers, they will reside together on the same physical processor or multiprocessor. The number of thread blocks can, however, greatly exceed the number of processors. The CUDA thread programming model virtualizes the processors and gives the programmer the flexibility to parallelize at whatever granularity is most convenient. Virtualization

into threads and thread blocks allows intuitive problem decompositions, as the number of blocks can be dictated by the size of the data being processed rather than by the number of processors in the system. It also allows the same CUDA program to scale to widely varying numbers of processor cores.

To manage this processing element virtualization and provide scalability, CUDA requires that thread blocks be able to execute independently. It must be possible to execute blocks in any order, in parallel or in series. Different blocks have no means of direct communication, although they may *coordinate* their activities using **atomic memory operations** on the global memory visible to all threads—by atomically incrementing queue pointers, for example. This independence requirement allows thread blocks to be scheduled in any order across any number of cores, making the CUDA model scalable across an arbitrary number of cores as well as across a variety of parallel architectures. It also helps to avoid the possibility of deadlock. An application may execute multiple grids either independently or dependently. Independent grids may execute concurrently, given sufficient hardware resources. Dependent grids execute sequentially, with an implicit interkernel barrier between them, thus guaranteeing that all blocks of the first grid complete before any block of the second, dependent grid begins.

Threads may access data from multiple memory spaces during their execution. Each thread has a private **local memory**. CUDA uses local memory for thread-private variables that do not fit in the thread's registers, as well as for stack frames and register spilling. Each thread block has a **shared memory**, visible to all threads of the block, which has the same lifetime as the block. Finally, all threads have access to the same **global memory**. Programs declare variables in shared and global memory with the `__shared__` and `__device__` type qualifiers. On a Tesla architecture GPU, these memory spaces correspond to physically separate memories: per-block shared memory is a low-latency on-chip RAM, while global memory resides in the fast DRAM on the graphics board.

Shared memory is expected to be a low-latency memory near each processor, much like an L1 cache. It can therefore provide high-performance communication and data sharing among the threads of a thread block. Since it has the same lifetime as its corresponding thread block, kernel code will typically initialize data in shared variables, compute using shared variables, and copy shared memory results to global memory. Thread blocks of sequentially dependent grids communicate via global memory, using it to read input and write results.

Figure B.3.5 shows diagrams of the nested levels of threads, thread blocks, and grids of thread blocks. It further shows the corresponding levels of memory sharing: local, shared, and global memories for per-thread, per-thread-block, and per-application data sharing.

A program manages the global memory space visible to kernels through calls to the CUDA runtime, such as `cudaMalloc()` and `cudaFree()`. Kernels may execute on a physically separate device, as is the case when running kernels on the GPU. Consequently, the application must use `cudaMemcpy()` to copy data between the allocated space and the host system memory.

atomic memory operation

A memory read, modify, write operation sequence that completes without any intervening access.

global memory Per-application memory shared by all threads.

shared memory Per-block memory shared by all threads of the block.

local memory Per-thread local memory private to the thread.

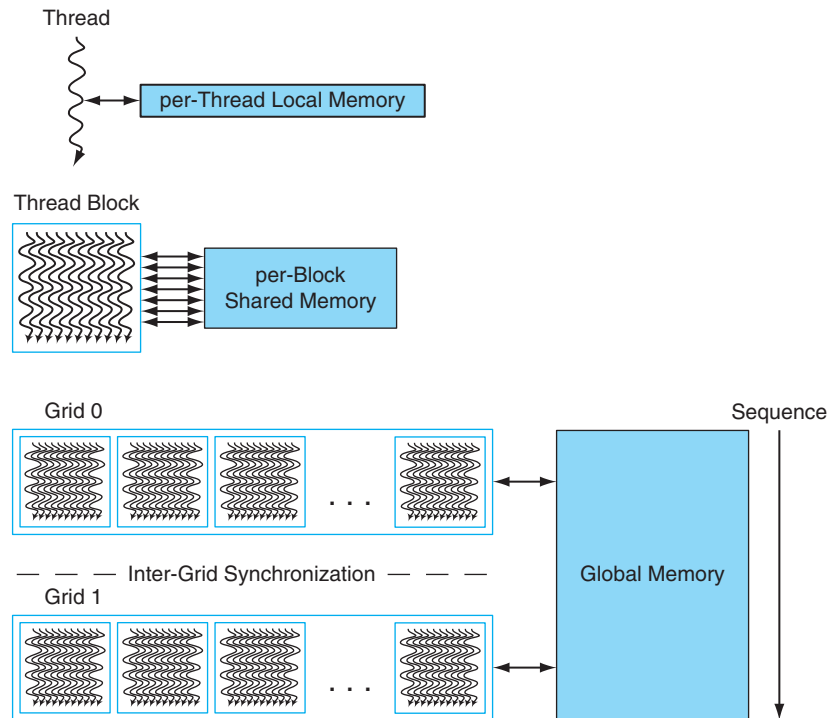


FIGURE B.3.5 Nested granularity levels—thread, thread block, and grid—have corresponding memory sharing levels—local, shared, and global. Per-thread local memory is private to the thread. Per-block shared memory is shared by all threads of the block. Per-application global memory is shared by all threads.

single-program multiple data (SPMD) A style of parallel programming model in which all threads execute the same program. SPMD threads typically coordinate with barrier synchronization.

The CUDA programming model is similar in style to the familiar **single-program multiple data (SPMD)** model—it expresses parallelism explicitly, and each kernel executes on a fixed number of threads. However, CUDA is more flexible than most realizations of SPMD, because each kernel call dynamically creates a new grid with the right number of thread blocks and threads for that application step. The programmer can use a convenient degree of parallelism for each kernel, rather than having to design all phases of the computation to use the same number of threads. Figure B.3.6 shows an example of an SPMD-like CUDA code sequence. It first instantiates `kernelF` on a 2D grid of 3×2 blocks where each 2D thread block consists of 5×3 threads. It then instantiates `kernelG` on a 1D grid of four 1D thread blocks with six threads each. Because `kernelG` depends on the results of `kernelF`, they are separated by an interkernel synchronization barrier.

The concurrent threads of a thread block express fine-grained data parallelism and thread parallelism. The independent thread blocks of a grid express coarse-grained data parallelism. Independent grids express coarse-grained task parallelism. A kernel is simply C code for one thread of the hierarchy.

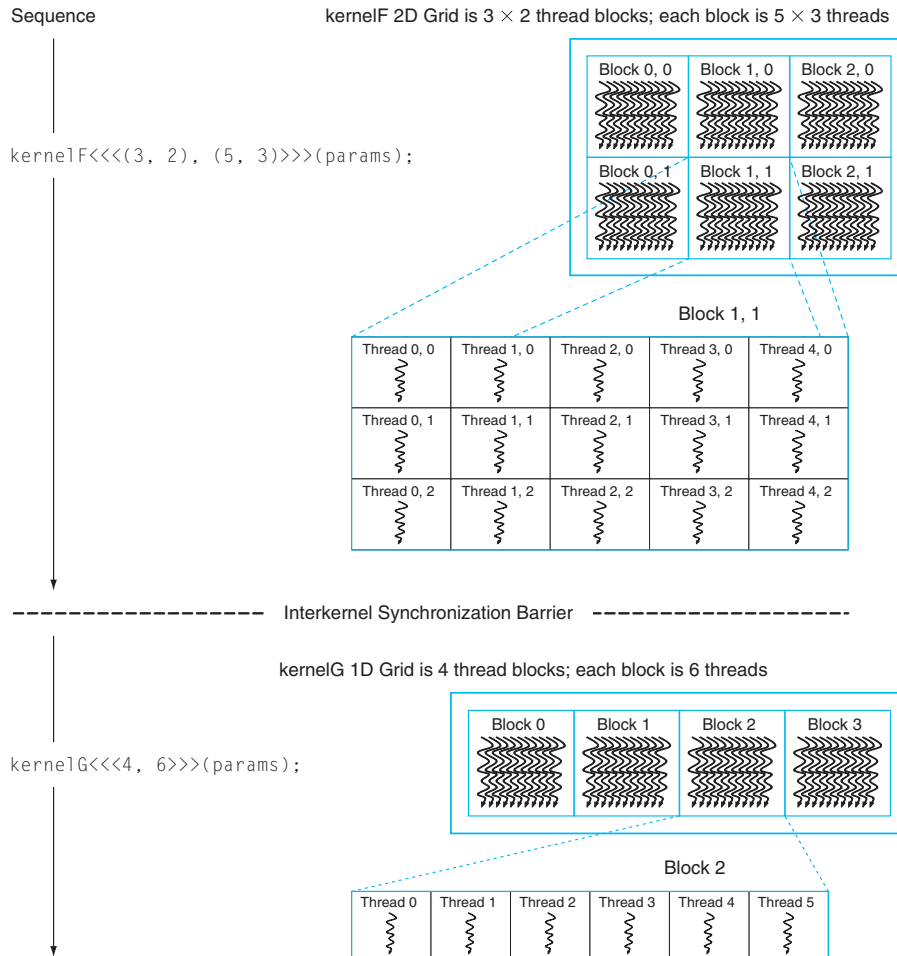


FIGURE B.3.6 Sequence of kernel *F* instantiated on a 2D grid of 2D thread blocks, an interkernel synchronization barrier, followed by kernel *G* on a 1D grid of 1D thread blocks.

Restrictions

For efficiency, and to simplify its implementation, the CUDA programming model has some restrictions. Threads and thread blocks may only be created by invoking a parallel kernel, not from within a parallel kernel. Together with the required independence of thread blocks, this makes it possible to execute CUDA programs with a simple scheduler that introduces minimal runtime overhead. In fact, the Tesla GPU architecture implements *hardware* management and scheduling of threads and thread blocks.

Task parallelism can be expressed at the thread block level but is difficult to express within a thread block because thread synchronization barriers operate on all the threads of the block. To enable CUDA programs to run on any number of processors, dependencies among thread blocks within the same kernel grid are not allowed—blocks must execute independently. Since CUDA requires that thread blocks be independent and allows blocks to be executed in any order, combining results generated by multiple blocks must in general be done by launching a second kernel on a new grid of thread blocks (although thread blocks may *coordinate* their activities using atomic memory operations on the global memory visible to all threads—by atomically incrementing queue pointers, for example).

Recursive function calls are not currently allowed in CUDA kernels. Recursion is unattractive in a massively parallel kernel, because providing stack space for the tens of thousands of threads that may be active would require substantial amounts of memory. Serial algorithms that are normally expressed using recursion, such as quicksort, are typically best implemented using nested data parallelism rather than explicit recursion.

To support a heterogeneous system architecture combining a CPU and a GPU, each with its own memory system, CUDA programs must copy data and results between host memory and device memory. The overhead of CPU–GPU interaction and data transfers is minimized by using DMA block transfer engines and fast interconnects. Compute-intensive problems large enough to need a GPU performance boost amortize the overhead better than small problems.

Implications for Architecture

The parallel programming models for graphics and computing have driven GPU architecture to be different than CPU architecture. The key aspects of GPU programs driving GPU processor architecture are:

- *Extensive use of fine-grained data parallelism:* Shader programs describe how to process a single pixel or vertex, and CUDA programs describe how to compute an individual result.
- *Highly threaded programming model:* A shader thread program processes a single pixel or vertex, and a CUDA thread program may generate a single result. A GPU must create and execute millions of such thread programs per frame, at 60 frames per second.
- *Scalability:* A program must automatically increase its performance when provided with additional processors, without recompiling.
- *Intensive floating-point (or integer) computation.*
- *Support of high-throughput computations.*